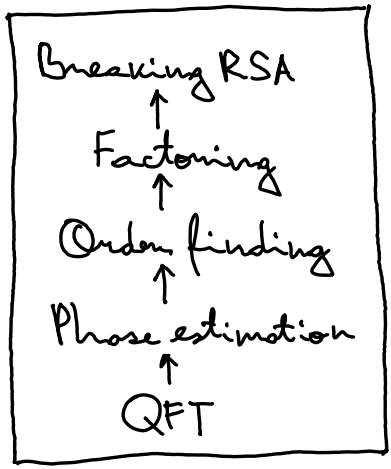
Lecture 11

Recall that in the previous class we opened the two bottom doors of our advent calendar – the one on quantum Fourier transform and phase estimation. Now we will open the next two – order finding and factoring (the top one, breaking RSA, you can open on Christmas yourself and read up on in a book).



To understand the order finding problem, we need to recap some basic number theory – modular arithmetic.

Extended Euclidean algorithm

It can be used to find the greatest common divisor and the modular multiplicative inverse.

Let $a, b \in \mathbb{Z}$ be integers (one of them not 0). The <u>greatest common divisor</u> of a and b, $\gcd(a,b)$, is the longest positive integer that divides both a and b.

Assume $a \geq b \geq 0$. Note that $\gcd(a,b) = \gcd(a-b, b)$. In fact, $\gcd(a,b) = \gcd(a - mb, b)$ for any $m \in \mathbb{Z}$. Let's take m as large as possible such that $a - mb \geq 0$, i.e. $m = \lfloor a/b \rfloor$ (or $a // b$ in Python notation). Then $a - mb$ is the remainder of a modulo b, i.e., $a \bmod b$ (or $a \,\%\, b$ in Python).

The Euclidean algorithm consists in repeatedly applying the substitution

$$(a, b) := (b, a \,\%\, b)$$

($\%$ = mod)

until the smallest integer becomes 0, which happens very fast. At this point a is equal to gcd of the original two numbers.

Example ($a=66$, $b=15$):

$$66 \bmod 15 = 6$$
$$15 \bmod 6 = 3$$
$$6 \bmod 3 = 0 \quad \gcd(66,15)=3$$

$$(a_k, b_k) := (b_{k-1}, a_{k-1} \bmod b_{k-1})$$

The extended version of Euclidean algorithm finds a solution to Bézout's identity by also keeping track of the quotients. Namely, it finds integers $x, y \in \mathbb{Z}$ such that

$$x \cdot a + y \cdot b = d$$

where $d = \gcd(a, b)$. Once we find the gcd, we have an equation $a_k - m_k \cdot b_k = d$, which expresses d as an integer linear combination of a_k and b_k. We can substitute b_k from the previous equation $a_{k-1} - m_{k-1} \cdot b_{k-1} = b_k$ to express d as an integer linear combination of a_{k-1} and $b_{k-1} = a_k$. We repeat until we get back to a_1 and b_1.

Example ($a=66$, $b=15$):

$$\underline{66} - 4 \cdot \underline{15} = \underline{6} \qquad \underline{15} - 2 \cdot (\underline{66} - 4 \cdot \underline{15}) = \overbrace{-2 \cdot \underline{66}}^{132} + \overbrace{9 \cdot \underline{15}}^{135} = \underline{3}$$

$$\underline{15} - 2 \cdot \underline{6} = \underline{3} = \gcd(a,b) \qquad x=-2 \quad y=9$$

Multiplicative inverse

If $\gcd(a,b)=1$, the integers a and b have no non-trivial common divisor and one called relatively prime. In such case Bézout's identity reads

$$x \cdot a + y \cdot b = 1.$$

If we take both sides modulo a, we get

$$y \cdot b \equiv 1 \pmod{a},$$

meaning y is the modular multiplicative inverse of b modulo a. We can write this as $y = b^{-1}$.

Example ($a = 66/3 = 22$, $b = 15/3 = 5$): Dividing the previous Bézout identity $-2 \cdot 66 + 9 \cdot 15 = 3$ by 3 we get $-2 \cdot 22 + 9 \cdot 5 = 1$. Modulo 22 gives $9 \cdot 5 \equiv 1 \pmod{22}$, so $5^{-1} \equiv 9 \pmod{22}$. Indeed, $5 \cdot 9 = 45 \equiv 1 \pmod{22}$.

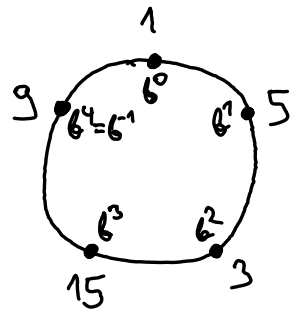
Multiplicative order

Hence, if $\gcd(a,b)=1$, we can both multiply and divide by b modulo a (on the other way around). What numbers do we get in this way? I.e., how does the set $\{b^k \bmod a : k \in \mathbb{Z}\}$ look like?

Example ($a=22$, $b=5$):

b^0	b^1	b^2	b^3	b^4	b^5	b^6	...
1	5	3	15	9	1	5	...

Hence, $b^k = b^{k+5}$, for any $k \in \mathbb{Z}$.



Let $a, N \in \mathbb{Z}$ with $N \geq 1$ and $\gcd(a,N)=1$. The multiplicative order of a modulo N is the smallest positive integer r such that $a^r \equiv 1 \pmod{N}$.

The reason such r always exists when $\gcd(a,N)=1$ is because there are at most N possible values that a^k can take modulo N. Hence, the sequence $a^0, a^1, a^2, \ldots$ has to repeat at some point, i.e.,

$$a^k \equiv a^{k+r} \pmod N,$$

for some $r \geq 1$ and all sufficiently large k. But since a is invertible, we can multiply both sides by $(a^{-1})^k = a^{-k}$ and get $1 \equiv a^r \pmod N$.

This also follows from a more general result known as Euler's Theorem: if a and N are relatively prime then

$$a^{\varphi(N)} \equiv 1 \pmod N,$$

where $\varphi(N)$ is the number of integers $c \in \{1, \ldots, N\}$ such that $\gcd(a,c)=1$. The security of the RSA cryptosystem is based on the assumption that it is hard to compute $\varphi(N)$ without knowing the prime factorization of N.

Factoring and order finding problems

Factoring: Assume $N=pq$ where p and q are odd primes (the general case can be handled with a little more effort).

- Input: N
- Output: p and q

Order finding: Assume $N \geq 2$ is an integer and $a \in \{2, \ldots, N-1\}$ is such that $\gcd(N,a)=1$.

- Input: a and N
- Output: the smallest integer $r \geq 1$ such that $a^r \equiv 1 \pmod N$, i.e., the order of a modulo N.

How to factor using order finding

Factor(N):

$a \in_R \{2, \dots, N-1\}$

$g = \gcd(N, a)$

if $g \neq 1$

return $(g, N/g)$ ← We were extremely lucky and guessed a factor

else

$r = \text{Order}(a, N)$ ← Call the quantum subroutine for order finding

if r is even, let $x = a^{r/2}$ else restart

if $x + 1 \not\equiv 0 \pmod N$

return $(\gcd(N, x+1), \gcd(N, x-1))$

else restart

Analysis of the algorithm:

Fact: If $N = pq$ and $a \in_R \{2, \dots, N-1\}$ is such that $\gcd(N, a) = 1$ then with probability $> 1/2$

- $\text{Order}(a, N)$ is even AND
- $a^{r/2} + 1 \not\equiv 0 \pmod N$

In other words, each run fails and has to restart with probability $< 1/2$. If we try k times, all k attempts fail with probability $< 2^{-k}$ which is extremely small.

If we made it till the very end, why are

$\gcd(N, x+1)$ and $\gcd(N, x-1)$

the factors of N?

Let $N = pq$, where p and q are odd primes, and assume we guessed an $a \in \{2, \dots, N-1\}$ such that

- $r = \text{Order}(a, N)$ is even AND
- $a^{r/2} + 1 \not\equiv 0 \pmod N$.

Let $x = a^{r/2}$. Since r is the order of a,

$$a^r \equiv 1 \pmod N$$

which is the same as $x^2 - 1 \equiv 0 \pmod N$. We can factor this polynomial as

$$(x-1)(x+1) \equiv 0 \pmod N. \qquad (*)$$

Note that

$x - 1 \not\equiv 0 \pmod N$ by minimality of r,

$x + 1 \not\equiv 0 \pmod N$ by the 2nd assumption.

So the condition $(*)$ is equivalent to

$$(x-1)(x+1) = k\,pq,$$

for some integer k. Since neither $x-1$ or $x+1$ is a multiple of N, and $N = pq$, computing

$$\gcd(N, x-1) \quad \text{and} \quad \gcd(N, x+1)$$

will find p and q. Either $p \mid (x-1)$ and $q \mid (x+1)$ or the other way round.

Order finding = Period finding

An alternative way to look at the problem of finding the order of a modulo N, i.e., computing $\text{Order}(a, N)$ is to find the period of the sequence

$$a^0, a^1, a^2, a^3, \dots \pmod N$$

Fourier transform and phase estimation are the right tools for this.

Let us now connect the period / order finding problem to what we discussed in the previous lesson.

Consider the function $f_a : \mathbb{N} \to \{0, \ldots, N-1\}$ given by

$$f_a(x) = a^x \mod N.$$

Note that f_a is periodic, with period at most N. Also, note that

$$f_a(0) = f_a(k)$$

is equivalent to

$$a^k \equiv 1 \pmod N,$$

so the period of f_a is equal to the order of a.

Period finding

Let us consider the more general problem of finding the period of some arbitrary function $f : \mathbb{N} \to \{0, \ldots, N-1\}$ which is promised to be periodic, i.e., there is some $r \geq 1$ such that

$$f(x+r) = f(x) \qquad \forall x.$$

We have the ability to evaluate this function in superposition on any input.

In the case of f_a, we can use a reversible classical circuit for implementing a^x (this can be done using repeated squaring and integer multiplication based on the binary digits of x). More generally, we can imagine having an oracle U_f for computing f and work from there.

Algorithm

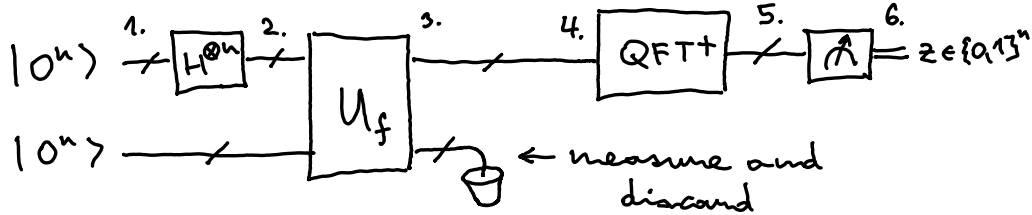
truncate $\mathbb{N}$ to n bits

$N = 2^n$

Let $f : \{0,1\}^n \to \{0,1\}^n$ and U_f be its bit oracle

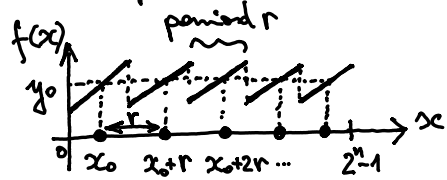


The quantum circuit is just like for Simon's algorithm, but with the second Hadamard transform $H^{\otimes n}$ replaced by QFT^+. It is also identical to the phase estimation circuit.



1. $|0^n\rangle|0^n\rangle$
2. $|+\rangle^{\otimes n}|0^n\rangle$
3. $\frac{1}{\sqrt{2}} \sum_{x=0}^{2^n-1} |x\rangle|f(x)\rangle$ ← the uniform superposition over all x and the corresponding $f(x)$
4. Let $y_0 \in \{0,1\}^n$ be the measurement outcome when measuring the 2nd register. The post-measurement state is

$$\left(\frac{1}{\sqrt{m}} \sum_{k=0}^{m-1} |x_0 + kr\rangle\right)|y_0\rangle$$



where

- $x_0 \in \{0, \dots, N-1\}$ is the least value s.t. $f(x_0) = y_0$,
- $r \in \{1, \dots, N-1\}$ is the period of f,
- $m = \lfloor 2^n / r \rfloor$ is the number of x s.t. $f(x) = y_0$.

The state in the first register has periodic structure while y_0 itself is not useful for determining r.

4. $\frac{1}{\sqrt{m}} \sum_{k=0}^{m-1} |x_0 + kr\rangle$

5. Recall that, for any $x \in \{0, \dots, 2^n - 1\}$,

$$QFT^+ |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z=0}^{2^n-1} \omega^{-xz} |z\rangle$$

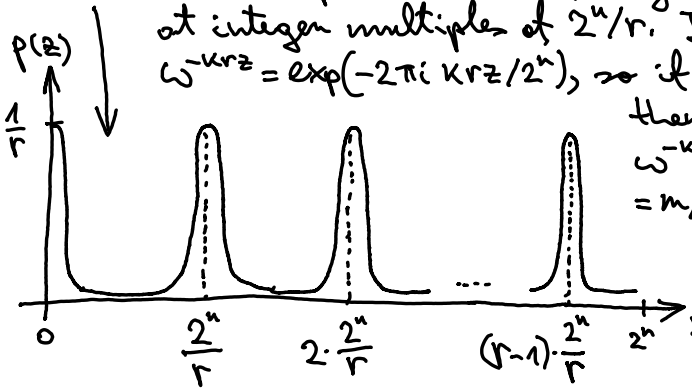
where $\omega = \exp(2\pi i / 2^n)$. Applying this on the state from the previous step gives

$$\frac{1}{\sqrt{m}} \sum_{k=0}^{m-1} QFT^+ \cdot |x_0 + kr\rangle$$
$$= \frac{1}{\sqrt{m}} \sum_{k=0}^{m-1} \frac{1}{\sqrt{2^n}} \sum_{z=0}^{2^n-1} \omega^{-(x_0+kr)z} |z\rangle$$
$$= \sum_{z=0}^{2^n-1} \omega^{-x_0 z} \frac{1}{\sqrt{m}} \cdot \frac{1}{\sqrt{2^n}} \left(\sum_{k=0}^{m-1} \omega^{-krz} \right) |z\rangle$$

6. The probability of observing outcome $z \in \{0,1\}^n$ in the final measurement is just the $|\cdot|^2$ of the amplitude in front of $|z\rangle$:

$$p(z) = \frac{1}{m 2^n} \left| \sum_{k=0}^{m-1} \omega^{-krz} \right|^2$$

The plot of $p(z)$, for $z \in \{0, \dots, 2^n - 1\}$, looks something like this. It has r peaks, each of height roughly $1/r$, located at integer multiples of $2^n/r$. Indeed, note that $\omega^{-krz} = \exp(-2\pi i krz/2^n)$, so if $z = l \cdot 2^n/r$, for some $l \in \mathbb{Z}$, then $2\pi i krz/2^n = 2\pi i kl$ and $\omega^{-krz} = 1$ and $p(z) = \frac{1}{m2^n} \cdot m^2 = m/2^n \approx 1/r$ since $m = \lfloor 2^n/r \rfloor$.



How do we recover r from the measurement outcome z?

The general procedure involves finding a continued fraction expansion for $z/2^n$. Just to get the main idea, let us consider the very special case when the period r itself is a power of 2: $r = 2^u$, for some $u \in \{0, \dots, n\}$. Then

$$m = \lfloor 2^n/r \rfloor = \lfloor 2^{n-u} \rfloor = 2^{n-u} = 2^n/r \qquad (*)$$

and the probability to get measurement outcome $z = l \cdot 2^n/r$ is exactly $1/r$ for all $l \in \{0, \dots, r-1\}$ since

$$\begin{aligned} p(z) &= \frac{1}{m 2^n} \left| \sum_{\kappa=0}^{m-1} \omega^{-\kappa r z} \right|^2 \\ &= \frac{1}{m 2^n} \left| \sum_{\kappa=0}^{m-1} \exp\left(-\frac{2\pi i}{2^n} \cdot \kappa r \cdot l \cdot \frac{2^n}{r} \right) \right|^2 \\ &= \frac{1}{m 2^n} \left| \sum_{\kappa=0}^{m-1} \underbrace{\exp(-2\pi i \cdot \kappa \cdot l)}_{1} \right|^2 \\ &= \frac{1}{m 2^n} \cdot m^2 = \frac{m}{2^n} = 1/r \qquad \text{by } (*). \end{aligned}$$

Hence we do not get any other outcomes than $z = l \cdot 2^n/r$ for some $l \in \{0, \dots, r-1\}$. To estimate r from such z, note that $z/2^n \in [0,1)$ is given by

$$z/2^n = l/r = l/2^u$$

since we assumed that $r = 2^u$. Since $l \in \{0, \dots, r-1\}$ is uniformly random, with probability $1/2$ it is odd so the binary expansion of $z/2^n = l/2^u$ looks as follows:

$$l/2^u = 0.\overbrace{1011\,00 \dots 0}^{n \text{ bits}}$$
$$\underbrace{}_{u \text{ bits}}$$

where the last non-zero bit is at position u. From this u we can infer r since $r = 2^u$. The case of a general r is more complicated and requires a continued fraction expansion for $z/2^n$.

Another way to think about it, which does not use period finding but directly connects to phase estimation:

$$U|x\rangle = |ax \bmod N\rangle$$

is a permutation matrix, so it is diagonal in the Fourier basis and its eigenvalues are of the form $\exp(2\pi i\, k/\ell)$ where ℓ is the length of some cycle. In particular, $U|1\rangle = |a\rangle$, $U|a\rangle = |a^2\rangle$, etc. is the cycle that goes through all powers of a, so the corresponding eigenvalue is $e^{\frac{2\pi i}{r}}$ where r is the order of a modulo N.

We can get r by running phase estimation on $|1\rangle$ (which is not an eigenvector itself, but has overlap $1/r$ with all eigenvectors, such as the one that is a uniform superposition of all powers of a).

Note that $\sum_{i=0}^{2^n-1} |i\rangle\langle i| \otimes U^i$ is the bit oracle for $f_a(x) = a^x \pmod N$, so that's why the algorithm is the way it is and we are literally just using phase estimation.